

LLMKernelBench: Benchmarking Large Language Models on Software Vulnerability Detection in Linux Kernel

Arastoo Zibaeirad, Marco Vieira

University of North Carolina at Charlotte, Charlotte, NC, USA

Email: {azibaeir, marco.vieira}@charlotte.edu

Abstract—LLMs show promise for software vulnerability detection (SVD). However, their effectiveness in security -critical tasks remains poorly understood. This paper introduces LLMKernelBench, a comprehensive benchmark framework for evaluating LLM capabilities in SVD and patch verification (PV) using real-world Linux kernel vulnerabilities. Our framework includes two complementary datasets—a Primary Benchmark Dataset (PBD) with 314 vulnerabilities spanning 74 CWE types and a Leakage-Free Dataset (LFD) with 103 recent vulnerabilities spanning 19 CWE types to address data contamination concerns. Through systematic evaluation of six state-of-the-art LLMs (three code-specialized, three general-purpose), we reveal critical insights: (1) all models achieve surprisingly low CWE detection accuracy (best: 11.4% Top-5), with systematic bias toward memory-safety vulnerabilities while completely failing on cryptographic and configuration flaws; (2) models exhibit unexpected reverse bias, detecting patched code better than vulnerable code (average 13.5% gap), contradicting security-first assumptions; (3) providing explicit CVE/CWE context often degrades rather than improves performance, revealing fundamental reasoning limitations; and (4) increasing code abstraction does not universally hinder detection—some models actually improve with additional context. These findings expose fundamental gaps in LLMs’ security reasoning and establish LLMKernelBench as a rigorous foundation for advancing LLM-based vulnerability detection research.

Index Terms—Large Language Models, Software Security, Software Vulnerability Detection

I. INTRODUCTION

LLMs, such as OpenAI’s Codex [1], Meta’s Llama [2], [3], CodeBERT [4], DeepMind’s AlphaCode [5], Salesforce’s CodeGen [6] and CodeT5 [7], and PLBART [8], have shown remarkable capabilities in understanding programming languages [9]. These models have potential for assisting developers by providing code suggestions, test generation [10]–[12], code documentation [13], and automating various coding tasks. In fact, by employing advanced methods to train billions of parameters on data from large datasets [14]–[17], LLMs demonstrate significant capabilities in various areas [18]. These models show also great potential for automating SVD, making software more secure by improving efficiency, accuracy, and coverage in identifying and fixing vulnerabilities.

The number of identified and patched software vulnerabilities has surged in recent years, with over 29,000 Common Vulnerabilities and Exposures (CVEs) addressed in 2023, up

from 25,084 in 2022 and 20,153 in 2021 [19]. This trend underscores the need for automated SVD to keep up with rapidly evolving digital technologies. There are, however, several limitations and challenges in SVD from the perspective of both LLMs and traditional VD techniques, such as automatic program repair, fuzzing, and static analysis tools.

From the perspective of LLMs, one major challenge lies in understanding the inherent complexity of vulnerability patterns, which involve intricate interactions between multiple functions or modules within the entire system, rather than isolated code snippets. For instance, a vulnerability might only surface when a specific sequence of function calls is made, or when data flows through various parts of the code in a particular order. Detecting these interactions requires a deep understanding the different parts of the code and of their interdependencies.

A key problem with state-of-the-art LLMs is that they are trained on billions of lines of code without explicitly distinguishing between vulnerable and non-vulnerable code, which can lead to ineffective identification and prevention of software vulnerabilities. For instance, while many LLMs may be trained on data including the Linux kernel project and know the entire codebase, they lack awareness of previous versions of the Linux kernel that contained numerous vulnerabilities. Furthermore, from a software engineering perspective, it is crucial that these models can detect vulnerabilities in code that they have never seen before, such as new projects or entirely new codebases. To enhance their effectiveness in SVD, a dataset that differentiates between vulnerable code versions and their patched counterparts is necessary.

Traditional VD techniques also present significant challenges. APR faces issues with patch quality and generalization [20], [21]. Fuzzing struggles with achieving comprehensive coverage and detecting logic errors [22], [23]. Similarly, SATs often suffer from a high rate of false positives and difficulty in understanding dynamic behaviors [24]. These limitations highlight the need for development of more sophisticated approaches, including effectively leveraging LLMs for SVD, which in turn requires a better understanding of their performance.

Despite the potential of LLMs for vulnerability tasks, significant gaps exist in evaluation frameworks for SVD capabilities. Current datasets suffer from a pronounced imbalance wherein

vulnerable code blocks are substantially outnumbered by non-vulnerable segments, resulting in skewed performance metrics that fail to accurately represent model effectiveness [25]. Additionally, many evaluations are conducted in artificially constrained settings using synthetic code snippets, rather than real-world codebases. This restricts the realism and generalizability of the results, whereas incorporating diverse, real-world code would better reflect LLM performance in practical scenarios [26], [27].

We propose LLMKernelBench, a comprehensive benchmark framework to systematically evaluate pre-trained LLMs’ capabilities in SVD and patch verification (PV). Our framework assesses six state-of-the-art pre-trained LLMs using two carefully curated datasets comprising 417 real-world Linux kernel vulnerabilities across 74 CWE types. We focus on Linux kernel C code for its rich vulnerability history and central role in production systems. As the largest open-source project (30M LOC) powering Android, servers, and embedded devices, the kernel exhibits diverse, high-impact flaws. If models cannot generalize to it, they are unlikely to be ready for real-world deployment. Unlike training-focused datasets, LLMKernelBench prioritizes evaluation rigor by pairing vulnerable code with corresponding patches and addressing data contamination through a leakage-free subset. Our evaluation spans multiple detection scenarios: CWE ranking to assess security knowledge breadth, binary vulnerability identification to test fundamental detection capabilities, and targeted detection with explicit CVE/CWE context to examine reasoning depth.

Research Questions. We study the following questions:

- **RQ1: Binary VD/PV performance** — How well do LLMs classify vulnerable vs. patched code across non-targeted and targeted settings?
- **RQ2: Per-CWE performance** — Which CWE categories (and pillars) are detected or missed, and how consistent are models across CWEs?
- **RQ3: CWE ranking quality** — What is overall ranking accuracy (micro/macro, Top- k) and ranking quality (MRR, HPS)?
- **RQ4: Robustness and context** — How do code abstraction levels (function/file/multi-file) and dataset shift (PBD→LFD) affect performance and generalization?

Contributions. This work offers:

- **Quality-controlled kernel benchmark:** PBD/LFD with paired vulnerable–patched code, clean CWE labels (variant and pillar), commit metadata, and a leakage-free subset (2025 CVEs).
- **Unified evaluation protocol:** CWE ranking (Top- k , MRR, HPS) and binary VD/PV (with abstention), analyzed with micro/macro accuracy.
- **Comprehensive study of six LLMs:** Results across per-CWE categories, misclassification patterns, vulnerable vs. patched bias, and PBD→LFD generalization.
- **Actionable insights:** Limitations and guidance for deploying LLMs in security triage and for designing future security-aware benchmarks.

The remainder of this paper is structured as follows: **Section II** reviews related work and contrasts benchmarks (Table I). **Section III** presents the benchmark and evaluation protocol. **Section IV** reports findings for RQ1–RQ4. **Section V** discusses implications and limitations. **Section VI** summarizes threats to validity. Finally, **Section VII** concludes the paper.

II. BACKGROUND AND RELATED WORK

CWE Taxonomy Primer. The CWE system, developed by MITRE [28], provides a hierarchical taxonomy of software security weaknesses organized into four abstraction levels: Pillars (highest-level categories like CWE-664: Resource Control), Classes (broader weakness categories), Base weaknesses (specific vulnerability types), and Variants (implementation-specific manifestations). This hierarchical structure enables graduated assessment of security knowledge, where identifying a parent category (e.g., CWE-119: Memory Buffer Bounds) demonstrates partial understanding even when the specific variant (e.g., CWE-120: Buffer Copy) is misidentified.

Data Collection. Much prior work labels at the file level rather than the function level, which easily introduces errors and bias. REVEAL [26], PatchDB [29], D2A [30], and MegaVul [31] include every function from a file touched by a vulnerability-related commit, so functions the patch never changed are labeled non-vulnerable. This creates label noise [32], [33], duplication/near-duplication across versions, and strong class imbalance [25]. Even function-level datasets (Devign [34], BigVul, CVEfixes [35]) often do not strictly pair the exact before/after function bodies and commonly pull commits dominated by refactoring, tests, or configuration updates—changes that are not security fixes (i.e., “noise”). Broad, multi-project corpora (e.g., BigVul, PrimeVul) favor coverage over precision, which increases noise and duplicates and makes it harder for models to learn the real difference between vulnerable and patched code.

Benchmarking LLMs for VD. Table I overviews representative efforts. CodeXGLUE/Devign [34], [36] is large but lacks patches and CWE labels. SecurityEval [37] offers only 130 Python snippet pairs without contamination control. VulDetectBench [38] evaluates program-level C/C++ with 48 CWE inference but no leakage checks. UniVul/LLM4Vuln [39] provides multi-language function pairs with CWE summaries and only partial cutoff awareness. The 5K study [40] aggregates across datasets (partial PV, CWE-wise analysis), while SecLLMHolmes [41] probes robustness on 8 CWEs with contamination tests. Earlier works such as Ullah et al. [41] rely on synthetic/curated cases with limited CWE breadth, and Khare et al. [42] focus on isolated functions—both miss cross-component context and realistic patch effects.

In contrast, LLMKernelBench targets kernel-style vulnerabilities with quality over breadth (Table I): 314+103 real CVE-linked samples across 74 CWEs; paired vulnerable–patched code (function plus non-functional context); CWE-aware evaluation at variant and pillar levels; and a leakage-free subset using post-cutoff 2025 CVEs. Our commit-level extraction retains only line-modified functions and associated

TABLE I: VD benchmarks at a glance (concise comparison).

Benchmark (Year)	Scale	CWE	Pairs/PV	Leakage	Granularity	Lang
LLMKernelBench (2024/25)	314+103	74	✓	✓	Function+context	C
CodeXGLUE [36] (2019)	27,318	–	✗	✗	Func	C,C++
SecurityEval [37] (2022)	130	75	✓	✗	Snippet	Python
VulnDetectBench [38] (2024)	1000	48	✓	✗	Program	C,C++
UniVul/LLM4Vuln [39] (2025)	294	77/86	✓	partial	Func	Solidity, Java, C,C++
[40] (2025)	5,000	25	partial	✗	Function+project	C,C++, Java
SecLLMHolmes [41] (2024)	228	8	✓	✓	Func	C, Python

context, avoiding label noise from untouched code. PBD/LFD are designed as stress-test benchmarks that complement rather than replace broader multi-project corpora, which are more suitable for LLM training and fine-tuning.

III. BENCHMARKING APPROACH

In this section, we present our framework and methodology to evaluate the performance of LLMs in SVD and PV. We start by introducing LLMKernelBench framework in Section III-A. Section III-B details our dataset construction process. Section III-C defines metrics for measuring effectiveness in SVD and PV tasks. Finally, Section III-D presents our standardized prompt templates.

A. LLMKernelBench

LLMKernelBench is designed to assess pre-trained LLMs (see Table V for the models evaluated) on SVD and PV tasks, focusing on C code from the Linux kernel while being extensible to other programming languages. As illustrated in Figure 1, the framework is based on vulnerability meta-data and code pairs extracted from Linux CVE commits, and includes prompts designed for SVD and PV tasks. For detection, LLMKernelBench evaluates models in zero-shot settings on both vulnerability identification and PV using binary classification metrics (precision, recall, accuracy, F1) and ranking metrics. This approach ensures benchmark reliability through representativeness (diverse real-world vulnerabilities), adaptability (extendable methodology), portability (applicable across various LLMs), and fairness (standardized evaluation procedures).

B. Dataset

Our method systematically extracts pairs of vulnerable code blocks and their corresponding patched versions from real-world Linux kernel commits. We focus on the Linux kernel due to its rich vulnerability history and critical security importance. We collect commit hashes, CVE identifiers, and CWE classifications from well-known vulnerability databases including NVD [43], similar to established practices [44]. For each vulnerability-fixing commit, we retrieve the vulnerable code from its parent commit (pre-fix version) and the patched code from the commit itself (post-fix version).

Our extraction process identifies all security-related functions and non-functional elements (global variables, macros, type definitions, structs/unions, headers) that were directly modified in each commit. We include only functions with actual line-level changes, excluding unchanged code to avoid label noise. Non-functional elements are tracked separately

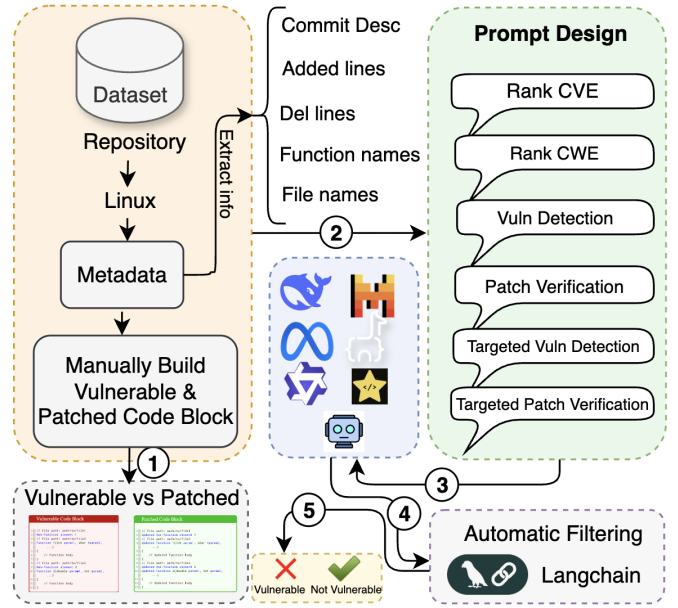


Fig. 1: LLMKernelBench Framework. ① Metadata (commit messages, diffs, function/file names) and vulnerable-patched code pairs are extracted from Linux CVE commits. ② Prompts are designed for SVD and PV tasks in zero-shot settings. ③ LLMs perform detection and PV. ④ Outputs are filtered via regex. ⑤ Detection is evaluated using binary and ranking metrics.

as they often form critical vulnerability contexts that influence execution across multiple functions. This comprehensive approach assembles cohesive vulnerable and patched code blocks at multiple abstraction levels (single function, multiple functions, multiple files), enabling LLMs to analyze complete contextual information rather than isolated fragments. Table III summarizes the overall size and composition of PBD and LFD.

Manual Review, Denoising, and Deduplication. Initially, we collected 400 samples for PBD and 110 samples for LFD through our automated extraction process. To ensure dataset integrity and label fidelity, both datasets underwent rigorous quality control through a structured two-author review process. Two authors independently evaluated every sample using a standardized protocol that defined inclusion criteria (security-relevant commits with confirmed CVE identifiers, function-level line modifications, and extractable vulnerable-patched pairs), exclusion criteria (cosmetic refactoring, formatting-only changes, test modifications, and commits lacking substantiated security relevance), and required evidence documentation (CVE/CWE identifiers, commit metadata, and security rationale). Each reviewer assigned decision labels (Keep, Exclude, Requires-Discussion) with documented reasoning codes to ensure traceability and reproducibility. Inter-reviewer agreement was measured using Cohen’s kappa, and all disagreements were resolved through consensus sessions with explicit rationale recording. Systematic denoising removed samples and touched functions/files that were not security-related,

eliminating non-functional changes while preserving essential contextual elements necessary for semantic correctness. We removed singleton CWEs (vulnerabilities appearing only once) to ensure statistical reliability and prevent overrepresentation of rare, potentially outlier cases that could skew model evaluation. Additionally, we stripped all code comments from both vulnerable and patched code blocks to eliminate potential biases that could arise from human annotations, ensuring LLMs rely solely on code structure and semantics for VD. Deduplication removed duplicate or near-duplicate code samples (e.g., identical functions, commits, or cloned files) to prevent models from seeing the same or highly similar code during testing, employing both exact hash-based matching and token-level similarity metrics with manual inspection of flagged candidates to ensure canonical representation. Each dataset contained only one multi-label CWE vulnerability, which we retained with comprehensive annotations rather than creating duplicate entries. For LFD, we manually labeled 40 samples that lacked CWE identifiers in the NVD database, strictly adhering to NVD criteria for consistent labeling. After this comprehensive cleaning process, our final datasets comprised 314 samples in PBD and 103 samples in LFD. The fundamental approach of LLMKernelBench assembles all touched functions and non-functional elements (macros, libraries, type definitions) into cohesive vulnerable and patched code blocks that LLMs can comprehensively analyze for vulnerability prediction, ensuring complete contextual understanding rather than isolated code fragment analysis.

CWE Distribution and Label Space. Our dataset spans diverse CWE categories organized according to the CWE-1000 hierarchical view. Table II shows the distribution of vulnerabilities across the ten fundamental CWE pillars in PBD. CWE-664 dominates with 43.3% coverage, reflecting the prevalence of memory-related vulnerabilities in the Linux kernel. The distribution also includes 12.1% deprecated CWE identifiers (CWE-OTHER), representing historical vulnerability classifications that MITRE now marks as "PROHIBITED" for mapping. This diverse distribution ensures comprehensive evaluation across different security weakness categories, from common memory safety issues to rare configuration vulnerabilities.

TABLE II: CWE Pillar Distribution in PBD

CWE-ID	Pillar Name	Coverage (%)
CWE-664	Improper Control of Resource Lifetime	43.3
CWE-OTHER	Other/Unmapped CWEs	12.1
CWE-682	Incorrect Calculation	10.2
CWE-284	Improper Access Control	8.9
CWE-691	Insufficient Control Flow Management	8.6
CWE-707	Improper Neutralization	5.7
CWE-693	Protection Mechanism Failure	5.1
CWE-710	Improper Adherence to Coding Standards	3.5
CWE-703	Improper Check/Handling of Exceptional Conditions	2.9
CWE-435	Improper Interaction Between Components	0.0
CWE-697	Incorrect Comparison	0.0

TABLE III: LLMKernelBench Dataset Statistics

Metric	PBD	LFD
Vulnerabilities	314	103
Unique CWEs	74	19
Files Changed (avg)	1.5	1.1
Functions Changed (avg)	2.0	1.3
Vulnerable Lines (avg)	109.4	60.9
Patched Lines (avg)	114.1	63.6
Lines Added (avg)	17.6	7.0
Lines Deleted (avg)	9.0	3.5

C. Metrics

We evaluate LLM performance on VD and PV tasks using established metrics selected for their relevance to security code analysis.

1) *Label Space and Task Formulation: Binary labels.* For all binary VD/PV tasks (SVD3–SVD6) we use labels $y \in \{1, 0\}$, where $y = 1$ denotes *vulnerable* code (the pre-fix version from the parent commit) and $y = 0$ denotes *patched* code (the post-fix version from the fixing commit). During inference we allow models to abstain with -1 ("not sure"); this is an optional prediction class and *not* a ground-truth label.

Why Pillar-Level Classification for LLMs? We focus on CWE pillars rather than classes, bases, or variants because LLMs perform better when classifying into fewer, more distinct categories—with over 900 specific CWE types, fine-grained classification suffers from the curse of dimensionality and class imbalance, while the 10 pillars provide optimal granularity that is specific enough to be actionable yet broad enough for reliable pattern recognition. Pillars represent fundamentally different security principles, making them more semantically distinct for neural networks (e.g., distinguishing between "access control failures" vs. "resource management issues"), and security practitioners often think in terms of root causes rather than specific manifestations, making pillar-level classification more valuable for remediation planning. Additionally, real-world vulnerability datasets exhibit severe imbalance at fine-grained levels, with many CWE variants having fewer than 10 examples, so pillar-level aggregation provides sufficient training examples for each category, enabling more robust model training and evaluation.

2) *SVD Metrics:* For SVD tasks, we employ:

- **Standard Classification Metrics:** Precision measures the ratio of correctly identified vulnerabilities to all predicted vulnerabilities. Recall quantifies the proportion of actual vulnerabilities correctly identified. F1 score provides a balanced measure between precision and recall. Accuracy represents overall correctness across all predictions.
- **Top-k Accuracy [45]:** Considers a prediction correct if the true vulnerability type appears among the model's k highest-ranked predictions. We measure Top-1 to Top-5 accuracy to evaluate how well models detect and rank both CVEs and CWEs. This progressive evaluation is particularly valuable in security contexts where identifying the correct vulnerability category among top candidates

enables more efficient remediation efforts, even when exact classification is challenging.

- **Mean Reciprocal Rank (MRR)** [46]: Evaluates ranking quality by calculating $\frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$, where rank_i is the position of the correct answer in the ranked list for the i -th query. MRR rewards models that rank the correct vulnerability type higher, recognizing that in security contexts, prioritization of findings is crucial.
- **Hierarchical Proximity Score (HPS)** scores a model's outputs by how close they are to the ground-truth CWE in the hierarchy. We assign weights: exact = 1.0, parent = 0.8, child = 0.7, sibling = 0.6, same-root = 0.4, unrelated = 0.0. Formally, let y_i be the ground truth and $(\hat{y}_{i1}, \dots, \hat{y}_{ik})$ the top- k predictions. Define $r(\hat{y}, y) \in \{\text{exact}, \text{parent}, \text{child}, \text{sibling}, \text{root}, \text{none}\}$ and a weight map $w(\cdot)$ as above. The per-sample score is

$$\text{HPS}_i = \max_{1 \leq j \leq k} w(r(\hat{y}_{ij}, y_i)),$$

and the dataset score is $\text{HPS} = \frac{1}{N} \sum_{i=1}^N \text{HPS}_i$. HPS credits near-misses that are semantically related, which is useful for prioritization in security triage.

D. Prompt Templates

Our prompt templates for VD and PV are designed with four key principles: clear task specification, structured output formats, minimal bias introduction, and scalability across vulnerability types. We focus on benchmarking inherent LLM capabilities rather than optimizing prompting strategies, evaluating baseline performance that can inform future research directions.

Each prompt clearly states the expected task and output format, uses standardized templates for fair comparison across models, maintains essential code context while avoiding information leakage, and reflects real-world security analysis scenarios. Our evaluation systematically assesses LLMs through ranking methods (SVD2) that identify the most likely vulnerability CWEs, and binary classification methods (SVD3-SVD6) that evaluate precise VD and PV capabilities.

Representative prompt formats for SVD2 and SVD3 are shown in Figure 2 and Figure 3.

TABLE IV: Prompt Templates for SVD and PV Tasks

ID	Task	Inputs				Description
		V	P	CVE	CWE	
SVD2	CWE Ranking	Y	-	-	-	Rank top-5 CWEs for given code.
SVD3	Detection	Y	-	-	-	Detect whether code is vulnerable (non-targeted).
SVD4	Verification	-	Y	-	-	Verify patched code is safe (non-targeted).
SVD5	Detection	Y	-	Y	Y	Detect vulnerability for a specific CVE/CWE (targeted).
SVD6	Verification	-	Y	Y	Y	Verify patch for a specific CVE/CWE (targeted).

V: vulnerable code (pre-fix); P: patched code (post-fix). All tasks use zero-shot prompts.

IV. BENCHMARKING RESULTS

In this study, we leverage six pre-trained LLMs to assess their capabilities in SVD and PV tasks. Our selection

CWE Ranking

Analyze this vulnerable code and return EXACTLY 5 CWE identifiers as a JSON list.

Code: {code}

Requirements:

1. Return ONLY a JSON array in this exact format: ["CWE-XXX", "CWE-YYY", "CWE-ZZZ", "CWE-AAA", "CWE-BBB"]
2. NO explanations, NO descriptions, NO additional text
3. Use only valid CWE identifiers (CWE- followed by numbers)
4. Order by likelihood (most likely first)

Response:

Fig. 2: CWE Ranking Prompt (SVD2)

Is Vulnerable?

Check if the following C code block is vulnerable. Respond with ONLY one of these options:

- '1' if the code is vulnerable
- '0' if the code is not vulnerable
- '-1' if you are not sure if it is vulnerable or not

Do not include any explanation or additional text.

Code block: {code_block}

Response:

Fig. 3: Is the code vulnerable (SVD3)

comprises three code-specialized models (CodeLlama [47], StarCoder2 [48], and Qwen3-Coder [49]) explicitly trained on programming repositories, and three general-purpose models (Llama3.1 [3], Mistral [50], and DeepSeek-R1 [51]) designed for broader language understanding tasks. This balanced selection enables us to investigate whether domain-specific training provides advantages over general language comprehension for security-critical tasks. We selected these models based on four key criteria: (1) their state-of-the-art performance on code understanding and generation benchmarks, (2) architectural diversity spanning encoder-decoder and decoder-only designs to ensure comprehensive evaluation, (3) varying parameter sizes (7B-32B) and context window capacities (16K-262K tokens) to investigate how model scale and memory affect VD capabilities, and (4) balanced representation of code-specialized versus general-purpose training to assess domain expertise effects. While proprietary models like GPT-4 and Claude are not included in this study, the considered open-source LLMs provide sufficient evidence to demonstrate and validate the framework, which is our primary goal. The specifications of the models we used are detailed in Table V.

A. RQ1: Binary Vulnerability vs. Patch Classification

We evaluate LLMs' ability to determine code vulnerability using precision, recall, accuracy, and F1 score metrics. Our binary classification framework assigns ground truth labels based on code state: vulnerable code blocks (pre-fix versions from parent commits) receive label 1, while patched code

TABLE V: LLM Specifications with Context Window Sizes (tokens)

Metric	Code-Spec.			General		
	CodeLlama	StarCoder2	Qwen3-Coder	Llama3.1	Mistral V0.3	DeepSeek-R1
# Parameters	7B	7B	32B	8B	7B	7B
Context Window	16K	16K	262K	131K	32K	131K

TABLE VI: LLM accuracy (%) on VD (SVD3–SVD6) across PBD and LFD. Arrows indicate performance change from PBD to LFD: $\uparrow$ improvement, $\downarrow$ degradation. **Bold**: best per task; Underlined: second-best. Values in parentheses show deviation from task mean. Bias Gap represents vulnerable vs. patched detection asymmetry.

Model	PBD					Bias Gap	LFD					Gen. Gap
	SVD3	SVD4	SVD5	SVD6	Avg		SVD3	SVD4	SVD5	SVD6	Avg	
CodeLlama	75.9 (+43.3)	21.2 (-22.0)	92.3 (+60.2)	11.3 (-36.1)	50.2	+67.8	70.9 (+42.3)	31.1 (-15.1)	92.2 (+62.7)	21.4 (-31.1)	53.9	$\uparrow$ 3.7
DeepSeek	11.2 (-21.5)	60.5 (+17.3)	41.4 (+10.3)	47.5 (+0.1)	40.1	-27.7	8.7 (-19.9)	63.1 (+16.9)	39.8 (+10.3)	55.3 (+2.8)	41.8	$\uparrow$ 1.6
Llama3.1	1.0 (-31.6)	42.4 (+9.9)	0.0 (-31.1)	46.5 (+9)	22.5	-44.0	0.0 (-38.0)	36.9 (-9.3)	0.0 (-29.5)	41.8 (-10.7)	19.7	$\downarrow$ 2.8
Mistral	88.2 (+55.0)	10.5 (-32.7)	20.1 (-11.0)	73.9 (+26.5)	48.2	+11.9	80.6 (+52.0)	21.4 (-24.8)	14.6 (-14.9)	82.5 (+30.0)	49.8	$\uparrow$ 1.6
Qwen3-Coder	3.3 (-29.3)	60.1 (+36.9)	0.7 (-30.4)	55.2 (+7.8)	29.8	-55.7	1.0 (-27.6)	56.9 (+10.7)	0.0 (-29.5)	57.8 (+5.3)	28.9	$\downarrow$ 0.9
StarCoder2	16.2 (-16.4)	64.7 (+23.5)	32.2 (+1.1)	50.0 (+2.0)	40.8	-33.1	10.7 (-17.9)	68.0 (+21.8)	30.1 (+0.0)	56.3 (+3.8)	41.3	$\uparrow$ 0.5
Avg	32.6	43.2	31.1	47.4	38.6	-13.5	28.6	46.2	29.5	52.5	39.2	$\uparrow$ 0.6

blocks (post-fix versions from fixing commits) receive label 0. To capture the inherent uncertainty in vulnerability assessment, we provide LLMs with a third option: -1 (meaning "not sure"). This uncertainty option is critical because: (1) real-world VD often involves ambiguous cases where even security experts disagree; (2) it prevents LLMs from making forced binary decisions when evidence is insufficient, potentially reducing false positives and negatives; (3) it provides insight into model confidence levels, enabling more nuanced evaluation of LLM reliability in security contexts; and (4) it mirrors practical security analysis workflows where analysts may need to flag code for further investigation rather than making definitive vulnerability determinations. Our analysis examines how models distinguish between these states across four prompts: SVD3 and SVD5 evaluate models on vulnerable code (expected output: 1), while SVD4 and SVD6 evaluate models on patched code (expected output: 0). SVD3/SVD4 assess non-targeted VD, while SVD5/SVD6 test targeted detection using specific CVE and CWE identifiers.

Overall Detection Performance and Model Rankings. Table VI reveals striking performance hierarchies across VD tasks. CodeLlama dominates with the highest overall accuracy on both PBD (50.2%) and LFD (53.9%), demonstrating exceptional performance on targeted vulnerable code detection (SVD5: 92.3% PBD, 92.2% LFD). Mistral follows closely (48.2% PBD, 49.8% LFD), excelling particularly at vulnerability identification tasks (SVD3: 88.2% PBD, 80.6% LFD) and PV with context (SVD6: 73.9% PBD, 82.5% LFD). However, the performance landscape shifts dramatically across tasks: StarCoder2 leads patch detection without context (SVD4: 64.7% PBD, 68.0% LFD), while Llama3.1 struggles catastrophically with near-zero vulnerable code detection (0.96% PBD SVD3, 0.0% LFD SVD3) yet maintains moderate patch identification capabilities (42.4% PBD SVD4), suggesting fundamental architectural weaknesses in security reasoning.

Vulnerability vs. Patch Detection Asymmetry and Bias Patterns. Models exhibit profound and systematic biases that contradict conventional security assumptions. While CodeLlama and Mistral demonstrate expected vulnerable-detection bias (CodeLlama: +67.8-point gap, Mistral: +11.9-point gap), four models show dramatic reverse bias: Qwen3-Coder (-55.7 points), Llama3.1 (-44.0 points), StarCoder2 (-33.1 points), and DeepSeek (-27.7 points). DeepSeek exemplifies this pattern, achieving only 26.3% vulnerable detection versus 54.0% patch detection in PBD, intensifying in LFD to 24.3% versus 59.2%. This reverse bias challenges the assumption that models default to flagging code as vulnerable, revealing instead that many models develop sophisticated patch-recognition heuristics while failing at vulnerability identification. The cross-dataset consistency (PBD average bias: -13.5%, LFD: -20.3%) confirms these reflect fundamental architectural tendencies rather than training artifacts, suggesting models learn to recognize "fixed" code patterns more effectively than "broken" ones.

CVE/CWE Context Utilization: Enhancement vs. Confusion. Explicit vulnerability identifiers produce divergent effects that reveal model-specific reasoning capabilities. DeepSeek benefits most dramatically from targeted context, improving from 35.8% (non-targeted) to 44.4% (targeted) in PBD (+8.6 points) and even more substantially in LFD (+11.7 points), suggesting sophisticated vulnerability-type reasoning. CodeLlama shows consistent moderate gains (+3.2 points PBD, +5.8 points LFD), indicating reliable context integration. Conversely, Mistral and Qwen3-Coder experience performance degradation when provided CVE/CWE information (Mistral: -2.4 points both datasets; Qwen3-Coder: -3.8 points PBD), revealing that explicit vulnerability identifiers overwhelm rather than guide these models' reasoning processes. The overall modest positive impact (PBD: +1.3%, LFD: +3.6%) masks this high variance, demonstrating that effective vulnerability-type utilization remains architecture-dependent and requires specialized investigation. Consider CVE-2025-21762 (CWE-416 use-after-free) from LFD: the vulnerable `arp_xmit()` function lacks RCU protection when calling `dev_net(skb->dev)`, creating a race condition where the device structure could be freed while still in use (Figure 4). The patch adds `rcu_read_lock()` and `rcu_read_unlock()` around the critical section and switches to `dev_net_rcu()`. In this case, only **CodeLlama** correctly classified the vulnerable and patched versions across all four detection tasks (SVD3–SVD6), while every other model either misclassified the code state or responded with "not sure." This results in an overall incorrectness ratio of **83.3%** (five out of six models), confirming that most LLMs still fail the fundamental binary judgment of whether code is vulnerable or not—even when explicit contextual information and commit descriptions are provided. Such systematic misclassification highlights that accurate vulnerability recognition remains an unsolved capability gap rather than a prompting or dataset limitation.

Cross-Dataset Generalization and Memory Analysis. The

```

1 // Vulnerable version
2 void arp_xmit(struct sk_buff *skb)
3 {
4     NF_HOOK(NFPROTO_ARP, NF_ARP_OUT,
5         dev_net(skb->dev), NULL, skb, NULL, skb->dev,
6         arp_xmit_finish);
7 }
8
9 // Patched version (kernel commit a42b69f69216)
10 void arp_xmit(struct sk_buff *skb)
11 {
12     rcu_read_lock();
13     NF_HOOK(NFPROTO_ARP, NF_ARP_OUT,
14         dev_net_rcu(skb->dev), NULL, skb, NULL, skb->dev,
15         arp_xmit_finish);
16     rcu_read_unlock();
17 }

```

Fig. 4: CVE-2025-21762: Use-After-Free (UAF) Vulnerability in Linux Kernel’s ARP Implementation. The vulnerable code (top) lacks RCU protection, while the patched code (bottom) adds necessary RCU read locks to prevent UAF conditions.

PBD-to-LFD performance transitions provide definitive evidence against training data contamination while revealing temporal reasoning patterns. Remarkably, most models improve on unseen LFD data: CodeLlama (+3.7%), DeepSeek (+1.6%), Mistral (+1.6%), and StarCoder2 (+0.5%) all demonstrate positive generalization. Only Llama3.1 (−2.8%) and Qwen3-Coder (−0.9%) show degradation. Task-specific analysis reveals asymmetric temporal effects: vulnerable code detection declines modestly (SVD3: −4.0%, SVD5: −1.6%) while patch detection consistently improves (SVD4: +3.0%, SVD6: +5.1%). This asymmetry indicates LFD’s temporal distribution (more recent vulnerabilities) better aligns with models’ training data for patch patterns, while historical vulnerable code styles in PBD may be overrepresented in pretraining corpora. The combination of low absolute LFD accuracies (best: 53.9%) with improved relative performance definitively rules out memorization, confirming that detection capabilities reflect learned security reasoning patterns rather than training data leakage.

B. RQ2: Per-CWE Performance and Pillars

Our comprehensive analysis across CWE root pillars (Figures 5 and 6) reveals dramatic specialization patterns and fundamental limitations in LLMs’ security reasoning capabilities. The results expose critical insights about how different models approach various vulnerability categories and their ability to generalize across rare and common security flaws.

Resource Control Dominance and Model Specialization. Table VII demonstrates that CWE-664 dominates both datasets with 136 vulnerabilities (43.3%) in PBD and 62 (60.2%) in LFD, representing memory-related vulnerabilities like buffer overflows, UAF, and resource leaks. CodeLlama

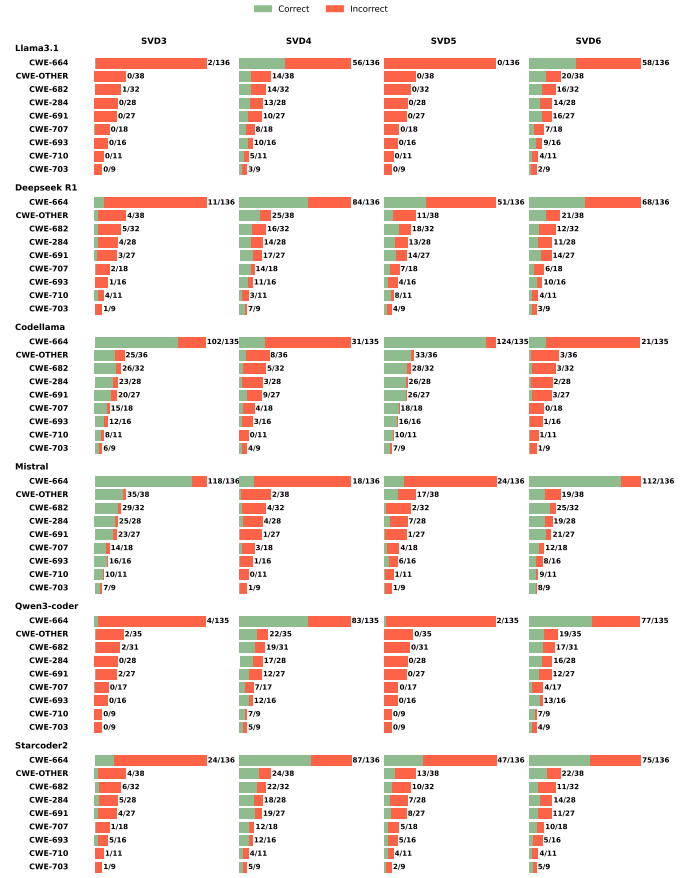


Fig. 5: Model Performance Across CWE Root Pillars in PBD: Distribution shows vulnerability counts by pillar with model accuracy rates.

TABLE VII: CWE Root Pillar Distribution and Model Specialization Patterns

Root Pillar	Dataset Coverage		Best Performing Models		
	PBD	LFD	Overall	Vulnerable	Patches
CWE-664: Resource Control	136 (43.3%)	62 (60.2%)	CodeLlama	Mistral	DeepSeek
CWE-OTHER: Deprecated/Unmapped	38 (12.1%)	0 (0.0%)	Mistral	Mistral	DeepSeek
CWE-682: Incorrect Calculation	32 (10.2%)	7 (6.8%)	CodeLlama	Mistral	StarCoder2
CWE-284: Improper Access Control	28 (8.9%)	0 (0.0%)	Mistral	Mistral	DeepSeek
CWE-691: Control Flow Management	27 (8.6%)	11 (10.7%)	Mistral	Mistral	StarCoder2
CWE-707: Improper Neutralization	18 (5.7%)	9 (8.7%)	CodeLlama	CodeLlama	StarCoder2
CWE-693: Protection Mechanism	16 (5.1%)	0 (0.0%)	Mistral	Mistral	Qwen3-Coder
CWE-710: Coding Standards	11 (3.5%)	14 (13.6%)	Mistral	Mistral	DeepSeek
CWE-703: Exception Handling	9 (2.9%)	0 (0.0%)	Mistral	Mistral	DeepSeek

Performance leadership based on highest combined accuracy across SVD3-SVD6 tasks. **Green:** VD specialist, **Blue:** PV specialist.

emerges as the dominant specialist for this category, achieving exceptional performance in targeted VD (SVD5: 91.9% in PBD, 91.9% in LFD), while struggling significantly with PV tasks. Conversely, Mistral excels across diverse vulnerability categories, showing remarkable consistency in vulnerability identification (SVD3: 86.8% for CWE-664, 100% for CWE-693) but moderate PV capabilities.

Deprecated CWE Challenge and Historical Bias. The presence of 38 CWE-OTHER vulnerabilities (12.1%) in

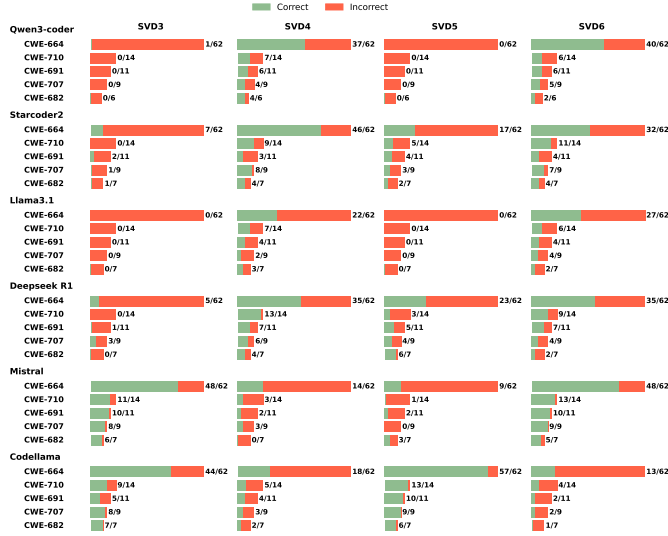


Fig. 6: Model Performance Across CWE Root Pillars in LFD.

TABLE VIII: Short-Tail vs. Long-Tail CWE Performance Analysis

Model	Accuracy (%)		Robustness Gap (Δ)	Frequency Correlation	
	Short-Tail	Long-Tail		r-value	Generalization
StarCoder2	40.7	40.6	+0.2	0.012	Excellent
Qwen3-Coder	29.6	28.9	+0.7	0.001	Excellent
Mistral	48.7	47.5	+1.2	0.122	Good
Llama3.1	22.0	22.4	-0.4	-0.066	Excellent
CodeLlama	50.4	51.0	-0.6	-0.076	Excellent
DeepSeek	39.5	40.9	-1.4	-0.218	Excellent
Average	38.5	38.6	-0.1	-0.037	—

Short-tail: top 20% most frequent CWEs; Long-tail: bottom 20% least frequent. Robustness Gap = Long-tail minus Short-tail accuracy. $|r| < 0.2$ denotes excellent generalization to rare types.

PBD—representing deprecated identifiers like CWE-189, CWE-264, and CWE-399—creates unique challenges. Mistral dominates this category with 92.1% VD accuracy, suggesting superior pattern recognition for historical vulnerability patterns that may no longer align with current security taxonomies. The complete absence of deprecated CWEs in LFD (0.0%) highlights the temporal evolution of vulnerability classification and potential training data bias in models exposed to historical security databases.

Tail Analysis and Frequency Independence. Table VIII reveals remarkable findings about LLMs’ generalization capabilities across vulnerability frequency distributions. Contrary to expectations, most models demonstrate frequency-independent performance: five of six models show excellent generalization ($|r| < 0.2$), with StarCoder2 achieving the most robust performance ($\Delta = +0.2\%$). The minimal average performance gap between short-tail (38.5%) and long-tail (38.6%) vulnerabilities contradicts conventional wisdom that LLMs perform better on frequent patterns. DeepSeek shows the strongest negative correlation ($r = -0.218$), actually

performing better on rare vulnerabilities, while Mistral exhibits the only concerning frequency bias ($r = 0.122$).

Task-Specific Vulnerability Domain Preferences. Analysis of individual SVD tasks reveals striking domain-specific capabilities that transcend simple frequency effects. CodeLlama demonstrates exceptional targeted detection capabilities across all vulnerability types (SVD5 tasks consistently $>85\%$), particularly excelling with CWE-707 at 100% accuracy in both datasets. However, this specialization comes at the cost of PV performance, where it consistently underperforms across all categories. DeepSeek shows the opposite pattern: moderate VD but superior patch identification capabilities, particularly for CWE-710 with 92.9% accuracy in LFD.

Dataset Distribution Effects and Temporal Patterns. The shift from PBD to LFD reveals fundamental changes in vulnerability landscape that significantly impact model performance. LFD’s increased concentration in CWE-710 from 3.5% to 13.6% and elimination of access control vulnerabilities (CWE-284: 8.9% to 0.0%) suggests evolving security practices in recent kernel development. Models show varying adaptation to these distribution shifts: Mistral maintains consistent leadership across both datasets, while others like Llama3.1 struggle universally, achieving near-zero performance in VD tasks across all categories.

These findings fundamentally challenge assumptions about LLM security capabilities, revealing that architectural differences and training methodologies create distinct vulnerability domain specializations rather than general security reasoning abilities. The frequency-independent performance suggests that current models have developed sophisticated pattern recognition for security concepts that transcends simple statistical frequency, though this capability remains highly architecture-dependent and task-specific.

C. RQ3: CWE Ranking Performance

To evaluate the effectiveness of pre-trained LLMs in SVD tasks, we conducted experiments using the CWE ranking task (SVD2) on our PBD and LFD. Table IX summarizes the performance of each model across various metrics, including Top-k accuracy, MRR, and HPS.

TABLE IX: CWE ranking (SVD2) across PBD and LFD. Columns report Top- k accuracy, MRR, and HPS. Parentheses show deviation (δ) from the column average; **bold** marks best.

Model	PBD: CWE Top-k (%)					LFD: CWE Top-k (%)					MRR (%)		HPS (%)	
	T_1	T_2	T_3	T_4	T_5	T_1	T_2	T_3	T_4	T_5	PBD	LFD	PBD	LFD
CodeLlama	0.64	1.27	1.27	1.27	1.27	1.96	2.94	2.94	2.94	6.86	1.03	3.40	5.83	7.06
DeepSeek	(-0.62)	(-1.41)	(-2.92)	(-4.06)	(-5.44)	(-0.72)	(-2.39)	(-2.39)	(-2.39)	(-4.97)	(-2.13)	(-2.15)	(-4.76)	(-7.61)
Llama3.1	0.64	1.61	2.57	3.22	3.86	0.00	1.00	1.00	2.00	2.00	1.74	0.75	5.16	9.62
Mistral	(-0.62)	(-1.07)	(-1.62)	(-2.11)	(-2.85)	(-2.68)	(-4.33)	(-4.33)	(-9.83)	(-9.83)	(-1.42)	(-4.80)	(-5.43)	(-5.05)
Qwen3-Coder	0.65	2.00	4.19	8.00	9.03	2.00	4.00	8.00	11.00	15.00	3.20	5.88	10.18	14.92
StarCoder2	(-0.61)	(-0.68)	(+0.00)	(+2.67)	(+2.32)	(-0.68)	(-1.33)	(+2.67)	(-0.59)	(+3.17)	(+0.04)	(+0.33)	(-0.41)	(+0.25)
StarCoder2	0.97	5.00	2.58	8.00	7.74	5.00	6.00	8.00	8.00	9.00	2.80	6.37	8.28	9.00
StarCoder2	(-0.29)	(+2.32)	(-1.61)	(+2.67)	(+1.03)	(+2.32)	(+0.67)	(+2.67)	(-3.59)	(-2.83)	(-0.36)	(+0.82)	(-2.31)	(-5.67)
StarCoder2	1.63	4.90	8.17	8.82	11.44	7.14	15.31	18.37	26.53	30.61	5.04	15.10	14.66	20.85
StarCoder2	(+0.37)	(+2.22)	(+3.98)	(+3.49)	(+4.73)	(-4.69)	(+9.98)	(+13.04)	(+15.94)	(+18.78)	(+1.88)	(+9.55)	(+4.07)	(+6.18)
StarCoder2	1.29	3.55	4.84	6.45	6.45	0.00	0.00	0.00	2.00	4.00	3.25	0.90	17.99	25.36
StarCoder2	(+0.03)	(+0.87)	(+0.65)	(+1.12)	(-0.26)	(-2.68)	(-5.33)	(-5.33)	(-9.59)	(-7.83)	(+0.09)	(-4.65)	(+7.40)	(+10.69)
Average	1.26	2.68	4.19	5.33	6.71	2.68	5.33	5.33	11.59	11.83	3.16	5.55	10.59	14.67

Overall Detection Capability and Micro/Macro Performance Disparity. Table X reveals significant performance

TABLE X: Micro/Macro accuracy analysis revealing model performance disparities across CWE types.

Model	PBD			LFD		
	Micro (%)	Macro (%)	Gap	Micro (%)	Macro (%)	Gap
CodeLlama	2.2	2.1	0.1	7.8	3.2	4.6
DeepSeek	3.9	3.2	0.7	2.0	1.8	0.2
Llama3.1	9.0	7.5	1.5	15.0	15.1	-0.1
Mistral	7.7	6.1	1.6	9.0	10.3	-1.3
Qwen3-Coder	11.4	8.7	2.7	30.6	21.3	9.3
StarCoder2	6.5	5.1	1.4	4.0	0.7	3.3

TABLE XI: CWE-specific detection performance showing memory safety dominance and universal blind spots.

Category	CWE (PBD)	Accuracy (%)	CWE (LFD)	Accuracy (%)
Best	CWE-119	53.3	CWE-20	41.7
	CWE-122	47.6	CWE-129	33.3
	CWE-20	45.6	CWE-787	22.2
	CWE-190	36.1	CWE-190	20.0
	CWE-129	26.7	CWE-476	14.4
Worst	CWE-327	0.0	CWE-416	6.8
	CWE-326	0.0	CWE-908	0.0
	CWE-732	0.0	CWE-667	0.0
	CWE-1284	0.0	CWE-401	0.0
	CWE-16	0.0	CWE-835	0.0

imbalances across vulnerability types. Qwen3-Coder achieves the highest Top-5 accuracy at 11.44% (Table IX) but exhibits substantial micro-macro gaps: 2.7 percentage points in PBD expanding to 9.3 points in LFD (30.6% vs 21.3%). This pattern indicates models overperform on common vulnerability classes rather than demonstrating uniform security reasoning. CodeLlama shows minimal PBD disparity (0.1 points) but significant LFD imbalance (4.6 points), confirming that current LLMs develop specialized heuristics for prevalent patterns rather than generalizable security knowledge.

CWE-Specific Bias and Memory Safety Dominance.

Table XI demonstrates pronounced specialization in memory-safety vulnerabilities. In PBD, CWE-119 achieves 53.3% detection, followed by CWE-122 at 47.6%, indicating strong pattern recognition for buffer-related issues. Conversely, cryptographic (CWE-327, CWE-326) and configuration vulnerabilities (CWE-732) achieve 0% detection, representing universal blind spots. LFD shows different patterns with CWE-20 leading at 41.7%, suggesting distribution shifts affect detection capabilities differently across vulnerability categories. This asymmetry reveals LLMs excel at syntactic vulnerabilities with clear code patterns but fail at semantic vulnerabilities requiring domain knowledge.

Model Consistency and Variance Analysis. Table XII highlights which CWEs produce consistent versus variable responses across models. Low-variance CWEs with 0% detection (CWE-1284, CWE-665, CWE-754) represent universal blind spots, indicating fundamental gaps rather than architecture-specific limitations. High-variance CWEs reveal model-dependent capabilities: CWE-119 shows 53.3% mean accuracy with $\sigma = 41.1$ (coefficient of variation 77%), meaning some models achieve near-perfect detection while others completely fail. Similar patterns in LFD (CWE-129: $\sigma = 47.1$, CWE-787: $\sigma = 36.9$) confirm these reflect inherent

TABLE XII: Model consistency analysis showing variance in CWE detection across different architectures.

Variance Level	CWE (PBD)	σ /Mean (%)	CWE (LFD)	σ /Mean (%)
High	CWE-119	41.1 / 53.3	CWE-129	47.1 / 33.3
	CWE-129	39.4 / 26.7	CWE-787	36.9 / 22.2
	CWE-399	37.3 / 16.7	CWE-20	34.4 / 41.7
Low	CWE-1284	0.0 / 0.0	CWE-667	0.0 / 0.0
	CWE-665	0.0 / 0.0	CWE-401	0.0 / 0.0
	CWE-754	0.0 / 0.0	CWE-835	0.0 / 0.0

TABLE XIII: Top misclassification patterns revealing systematic confusion toward CWE-119.

PBD			LFD		
True CWE	Predicted	Count	True CWE	Predicted	Count
CWE-276	CWE-119	13	CWE-416	CWE-119	52
CWE-665	CWE-119	12	CWE-125	CWE-119	26
CWE-667	CWE-119	12	CWE-416	CWE-123	20
CWE-416	CWE-119	12	CWE-416	CWE-121	19
CWE-120	CWE-119	12	CWE-416	CWE-20	18
CWE-399	CWE-119	12	CWE-476	CWE-119	16
CWE-122	CWE-119	12	CWE-416	CWE-400	17
CWE-862	CWE-119	12	CWE-416	CWE-1234	17

model differences rather than dataset artifacts.

Hierarchical Proximity and Semantic Understanding. HPS scores from Table IX provide deeper insight into models' security concept comprehension beyond exact CWE matching by awarding partial credit for predicting parent, child, or sibling categories within the CWE taxonomy (see Appendix). StarCoder2 achieves the highest HPS at 17.99% (PBD) and 25.36% (LFD) despite low Top-1 accuracy (1.29% and 0%), indicating semantically related predictions within the hierarchy. As a simple proxy for "informed guessing," the HPS-to-Top-1 ratio for StarCoder2 is 13.9. Full per-model details are in Table IX.

Generalization and Data Contamination Assessment. The PBD-to-LFD performance shift in Table IX provides critical insights into generalization and potential memorization. Surprisingly, most models show *negative* degradation (performance improvement): Mistral improves from 0.97% to 5.00% Top-1 accuracy (+4.03 percentage points), Qwen3-Coder from 1.63% to 7.14% (+5.51 points), CodeLlama from 0.64% to 1.96% (+1.32 points), and Llama3.1 from 0.65% to 2.00% (+1.35 points). This counterintuitive result indicates: (1) LFD's different CWE distribution (fewer rare types, more CWE-20/129) favors models' strengths, or (2) recent vulnerability patterns better match models' training distributions than historical ones. Only two models show positive degradation: StarCoder2 drops from 1.29% to 0% and DeepSeek from 0.64% to 0%, likely due to incidental exposure to PBD vulnerabilities during training. Critically, the low absolute Top-1 performance on LFD (best: 7.14%) combined with improved relative performance rules out significant data contamination while confirming that CWE detection capability is distribution-dependent rather than memorization-based.

Systematic Misclassification Patterns and Semantic Confusion. Table XIII reveals models consistently misattribute diverse vulnerability types to CWE-119, creating a "semantic

magnet" effect. In PBD, all top confusion pairs target CWE-119, including access control (CWE-276→CWE-119: 13 instances) and authorization issues (CWE-862→CWE-119: 12 instances). This intensifies in LFD with CWE-416→CWE-119 occurring 52 times. The diversity of source CWEs—spanning access control, resource management, and memory operations—all incorrectly mapped to CWE-119 indicates models lack fundamental understanding of security domains, defaulting to "buffer overflow" as an all-purpose prediction when code exhibits complexity.

D. RQ4: Effect of Code Abstraction Level

Our comprehensive analysis of six state-of-the-art LLMs across three abstraction levels reveals surprising patterns that challenge conventional assumptions about VD complexity. We evaluated both Primary Benchmark Dataset (PBD) and Leakage-Free Dataset (LFD) to ensure robust findings.

Abstraction Does Not Always Hinder Performance. Contrary to expectations, increasing code abstraction from single functions to multiple files does not universally degrade LLM performance. Statistical analysis shows no significant difference between abstraction levels in either dataset (PBD: $p=0.8559$, LFD: $p=0.4259$), with mean accuracies remaining remarkably stable: Level 1 (49.4%), Level 2 (48.7%), and Level 3 (48.8%) in PBD.

Model Architecture Matters More Than Size. Our robustness analysis in Table XIV reveals that model specialization and architecture significantly impact abstraction sensitivity. In PBD, Codellama emerges as the most robust model with performance actually *improving* by 2.6% from Level 1 to Level 3, while Starcoder shows the highest degradation at 5.1%. Surprisingly, general-purpose models (0.5% avg degradation) slightly outperform code-specialized models (0.6% avg degradation).

Context Window Size Shows Weak Correlation. We find no significant correlation between context window size and robustness (PBD: $r=0.678$, $p=0.322$; LFD: $r=-0.530$, $p=0.470$).

Task-Specific Patterns Emerge. Table XV demonstrates that different SVD tasks exhibit distinct sensitivity patterns. VD (SVD3) actually improves with abstraction (L1→L3: -6.1% degradation in PBD), while patch detection (SVD4) remains stable (1.0% degradation). This suggests LLMs benefit from additional context when identifying vulnerabilities but maintain consistent performance in distinguishing patches.

Leakage-Free Results Show Greater Variance. The LFD analysis reveals more dramatic performance differences, with degradation ranging from -6.9% (Deepseek improvement) to 22.0% (Codellama degradation). This higher variance (std=8.61 vs 3.01 in PBD) indicates that data leakage may have artificially stabilized performance metrics in traditional benchmarks.

Key Implications for VD Systems. Our findings fundamentally challenge the assumption that increased code abstraction universally hinders LLM performance. The lack of statistical significance across abstraction levels, combined with task-specific improvements, suggests that LLMs can effectively

TABLE XIV: Model Robustness Ranking by Abstraction Level Sensitivity

Rank	Model	Type	Context Window	Degradation (%)		
				PBD	LFD	Avg
1	Codellama	Code-Spec.	16K	-2.6	22.0	9.7
2	Deepseek	General	64K	-1.7	-6.9	-4.3
3	Qwen3	Code-Spec.	32K	-0.8	-0.3	-0.6
4	Llama	General	128K	1.4	0.0	0.7
5	Mistral	General	32K	1.9	0.0	1.0
6	Starcode	Code-Spec.	8K-16K	5.1	8.9	7.0
Code-Specialized Avg.				0.6	10.2	5.4
General-Purpose Avg.				0.5	-2.3	-0.9

Note: Degradation calculated as Level 1 accuracy minus Level 3 accuracy. Negative values indicate performance improvement. Color coding: **Green** = improvement, **Orange** = minor degradation (<3%), **Red** = significant degradation (≥5%).

TABLE XV: Task-Specific Performance Patterns Across Abstraction Levels

SVD Task	Description	PBD Accuracy (%)			LFD Accuracy (%)		
		L1	L2	L3	L1	L2	L3
SVD3	Vuln Detection	30.7	35.0	36.8	28.2	31.1	27.8
SVD4	Patch Detection	43.5	43.3	42.5	48.7	34.4	33.3
SVD5	Vuln + CWE	30.5	32.5	32.1	28.6	35.6	22.2
SVD6	Patch + CWE	47.3	44.9	49.1	52.7	48.9	66.7
Degradation	PBD (L1→L3)	-6.1	1.0	-1.7	-1.7		
	LFD (L1→L3)	0.5	15.4	6.4	-14.0		

Note: L1=Level 1, L2=Level 2, L3=Level 3. Color coding highlights best performance per task. Degradation shows % point change from L1 to L3.

leverage additional context when properly trained. The superior performance of general-purpose models over code-specialized ones in abstraction robustness indicates that broad language understanding may be more valuable than domain-specific training for handling complex code structures. Most importantly, the dramatic variance in leakage-free results highlights the critical need for rigorous evaluation methodologies that avoid data contamination, as traditional benchmarks may significantly overestimate model capabilities in real-world deployment scenarios.

V. DISCUSSION

Key Findings. Our benchmarking exposes fundamental limitations in LLMs' security reasoning. The low CWE detection accuracy (best: 11.4% Top-5) and systematic bias toward memory-safety vulnerabilities reveal surface-level pattern recognition rather than deep security understanding. The reverse bias—where models detect patched code 13.5% better than vulnerable code—contradicts assumptions about security-conscious defaults. Performance degradation with explicit CVE/CWE context suggests additional information overwhelms rather than guides reasoning. These findings indicate LLMs should serve as assistive tools under expert supervision, not autonomous analyzers, requiring significant architectural improvements for reliable vulnerability identification.

Future Directions. LLMKernelBench enables several research paths: (1) analyzing LLMs' reasoning processes to identify security comprehension weaknesses [52], [53]; (2)

TABLE XVI: Architectural Sensitivity Analysis: Context Window vs Model Type

Category	Models	Avg Degradation	Best Model	Worst Model	Correlation
By Context Window (PBD)					
Short ($\leq 16K$)	2	1.2%	Codellama (-2.6%)	Starcode (5.1%)	$r=0.678$
Medium (32K-64K)	3	-0.2%	Deepseek (-1.7%)	Mistral (1.9%)	$p=0.322$
Long ($\geq 128K$)	1	1.4%	Llama (1.4%)	Llama (1.4%)	(n.s.)
By Model Specialization					
Code-Specialized	3	0.6% (PBD)	Codellama (-2.6%)	Starcode (5.1%)	—
		10.2% (LFD)	Qwen3 (-0.3%)	Codellama (22.0%)	—
General-Purpose	3	0.5% (PBD)	Deepseek (-1.7%)	Mistral (1.9%)	—
		-2.3% (LFD)	Deepseek (-6.9%)	Llama (0.0%)	—
By Model Size (Estimated)					
Small ($\leq 7B$)	1	-1.7% (PBD)	Deepseek	Deepseek	—
Medium (8B-22B)	3	-0.5% (PBD)	Qwen3 (-0.8%)	Mistral (1.9%)	—
Large ($\geq 30B$)	2	1.3% (PBD)	Codellama (-2.6%)	Starcode (5.1%)	—

Note: Correlation analysis shows no significant relationship between context window size and robustness. General-purpose models demonstrate superior robustness, particularly in leakage-free conditions.

expanding to multiple languages (Python, Java, Rust, Go) to assess cross-language transfer; (3) investigating hybrid LLM-agent architectures integrating static/dynamic analysis tools; (4) examining whether LLMs introduce vulnerabilities when generating patches. These directions aim to transform our diagnostic framework into a catalyst for developing robust, security-aware models.

VI. THREATS TO VALIDITY

Internal Validity. We set temperature to 0 for all experiments to ensure deterministic outputs and reproducible results. Our benchmark evaluates fundamental VD capabilities using standardized zero-shot prompts rather than optimized prompt engineering, establishing baseline performance for future research. To prevent data contamination, we curated LFD exclusively from vulnerabilities disclosed in 2025. Since this date is after the models’ training cutoffs, their inclusion in the training corpora is highly unlikely. The two-author manual review with Cohen’s kappa measurement and consensus resolution ensures label quality and reduces annotation bias. All tasks are static code reasoning (no compilation). Snippets may be non-compilable and can include function bodies or non-function edits (e.g., macros, configuration gates, header guards); we include only edits judged security-relevant. We do not reconstruct full build systems (includes, Make/Kconfig).

External Validity. While focused on Linux kernel C code, our evaluation spans six diverse models (three code-specialized, three general-purpose) with varying architectures and parameter sizes, revealing consistent patterns across model types. The kernel’s complexity and widespread use in security-critical systems make it representative of real-world vulnerability detection challenges. Performance trends observed—memory-safety bias, reverse detection bias, context degradation—likely generalize to similar systems programming contexts.

VII. CONCLUSION

This paper presents LLMKernelBench, a rigorous benchmark for evaluating LLM capabilities in vulnerability detection and patch verification using 417 real-world Linux

kernel vulnerabilities. Our systematic evaluation of six state-of-the-art models reveals critical limitations: surprisingly low CWE detection accuracy (11.4% Top-5), systematic bias toward memory-safety vulnerabilities, unexpected reverse bias favoring patch over vulnerability detection (13.5% gap), and performance degradation with explicit security context. These findings challenge assumptions about LLMs’ security reasoning and demonstrate that current models require expert supervision rather than autonomous deployment. LLMKernelBench establishes a standardized foundation for advancing LLM-based security tools through reproducible evaluation and identification of specific capability gaps requiring architectural innovation.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [3] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [4] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, “Codebert: A pre-trained model for programming and natural languages,” *arXiv preprint arXiv:2002.08155*, 2020.
- [5] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, “Competition-level code generation with alphacode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [6] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, “Codegen: An open large language model for code with multi-turn program synthesis,” *arXiv preprint arXiv:2203.13474*, 2022.
- [7] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, “Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” *arXiv preprint arXiv:2109.00859*, 2021.
- [8] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, “Unified pre-training for program understanding and generation,” *arXiv preprint arXiv:2103.06333*, 2021.
- [9] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, “Using an llm to help with code understanding,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [10] N. Alshahwan, J. Chheda, A. Finegenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, “Automated unit test improvement using large language models at meta,” *arXiv preprint arXiv:2402.09171*, 2024.
- [11] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey, landscape, and vision,” *IEEE Transactions on Software Engineering*, 2024.
- [12] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering*, 2023.
- [13] J. Y. Khan and G. Uddin, “Automatic code documentation generation using gpt-3,” in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–6.
- [14] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, “Training compute-optimal large language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.15556>
- [15] P. Gholami and H. Seferoglu, “Improved generalization bounds for communication efficient federated learning,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.11754>

- [16] —, “Digest: Fast and communication efficient decentralized learning with local updates,” *IEEE Transactions on Machine Learning in Communications and Networking*, pp. 1–1, 2024.
- [17] S. Thomas, F. Kolehini, and N. Tabrizi, “Dynamic defenses and the transferability of adversarial examples,” in *2022 IEEE 4th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA)*. IEEE, 2022, pp. 276–284.
- [18] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: Survey and open problems,” in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 2023, pp. 31–53.
- [19] CVE Details, “Vulnerability details and information,” <https://www.cvedetails.com/>, 2024, accessed: 2024-06-19.
- [20] K. Huang, Z. Xu, S. Yang, H. Sun, X. Li, Z. Yan, and Y. Zhang, “A survey on automated program repair techniques,” *arXiv preprint arXiv:2303.18184*, 2023.
- [21] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, “A survey of learning-based automated program repair,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, pp. 1–69, 2023.
- [22] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, “The art, science, and engineering of fuzzing: A survey,” *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2019.
- [23] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, “Evaluating fuzz testing,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 2123–2138.
- [24] J. D. Pereira, J. R. Campos, and M. Vieira, “Machine learning to combine static analysis alerts with software metrics to detect security vulnerabilities: An empirical study,” in *2021 17th European Dependable Computing Conference (EDCC)*. IEEE, 2021, pp. 1–8.
- [25] Z. Gao, H. Wang, Y. Zhou, W. Zhu, and C. Zhang, “How far have we gone in vulnerability detection using large language models,” *arXiv preprint arXiv:2311.12420*, 2023.
- [26] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet?” *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.
- [27] Y. Wu, N. Jiang, H. V. Pham, T. Lutellier, J. Davis, L. Tan, P. Babkin, and S. Shah, “How effective are neural networks for fixing security vulnerabilities,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1282–1294.
- [28] MITRE Corporation, “Common Weakness Enumeration (CWE),” <https://cwe.mitre.org/>, 2024, accessed: 2025-10-18.
- [29] X. Wang, S. Wang, P. Feng, K. Sun, and S. Jajodia, “Patchdb: A large-scale security patch dataset,” in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2021, pp. 149–160.
- [30] Y. Zheng, S. Pujar, B. Lewis, L. Buratti, E. Epstein, B. Yang, J. Laredo, A. Morari, and Z. Su, “D2a: A dataset built for ai-based vulnerability detection methods using differential analysis,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 111–120.
- [31] C. Ni, L. Shen, X. Yang, Y. Zhu, and S. Wang, “Megavul: Ac/c++ vulnerability dataset with comprehensive code representations,” in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 738–742.
- [32] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, and F. Palomba, “The secret life of software vulnerabilities: A large-scale empirical study,” *IEEE Transactions on Software Engineering*, vol. 49, no. 1, pp. 44–63, 2022.
- [33] R. Croft, M. A. Babar, and M. M. Kholoosi, “Data quality for software vulnerability datasets,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 121–133.
- [34] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” *Advances in neural information processing systems*, vol. 32, 2019.
- [35] G. Bhandari, A. Naseer, and L. Moonen, “Cvefixes: automated collection of vulnerabilities and their fixes from open-source software,” in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021, pp. 30–39.
- [36] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, “Codexglue: A machine learning benchmark dataset for code understanding and generation,” *arXiv preprint arXiv:2102.04664*, 2021.
- [37] M. L. Siddiq and J. C. Santos, “Securityeval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques,” in *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022, pp. 29–33.
- [38] Y. Liu, L. Gao, M. Yang, Y. Xie, P. Chen, X. Zhang, and W. Chen, “Vuldetechbench: Evaluating the deep capability of vulnerability detection with large language models,” *arXiv preprint arXiv:2406.07595*, 2024.
- [39] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, M. Shi, and Y. Liu, “Llm4vuln: A unified evaluation framework for decoupling and enhancing llms’ vulnerability reasoning,” *arXiv preprint arXiv:2401.16185*, 2024.
- [40] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, “Understanding the effectiveness of large language models in detecting security vulnerabilities,” in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 103–114.
- [41] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, “Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks,” in *IEEE Symposium on Security and Privacy*, 2024.
- [42] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, “Understanding the effectiveness of large language models in detecting security vulnerabilities,” *arXiv preprint arXiv:2311.16169*, 2023.
- [43] N. I. of Standards and T. (NIST), “National vulnerability database (nvd) dashboard,” <https://nvd.nist.gov/general/nvd-dashboard>, 2025, accessed: 2025-04-24.
- [44] J. D. Pereira, J. H. Antunes, and M. Vieira, “A software vulnerability dataset of large open source c/c++ projects,” in *2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC)*. IEEE, 2022, pp. 152–163.
- [45] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, “Imagenet large scale visual recognition challenge,” *International journal of computer vision*, vol. 115, pp. 211–252, 2015.
- [46] N. Craswell, “Mean reciprocal rank,” *Encyclopedia of database systems*, pp. 1703–1703, 2009.
- [47] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [48] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei *et al.*, “Starcode 2 and the stack v2: The next generation,” *arXiv preprint arXiv:2402.19173*, 2024.
- [49] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [50] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, “Mistral 7b,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [51] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [52] V. Xiang, C. Snell, K. Gandhi, A. Albalak, A. Singh, C. Blagden, D. Phung, R. Rafailov, N. Lile, D. Mahan *et al.*, “Towards system 2 reasoning in llms: Learning how to think with meta chain-of-thought,” *arXiv preprint arXiv:2501.04682*, 2025.
- [53] A. Zibaeirad and M. Vieira, “Reasoning with llms for zero-shot vulnerability detection,” *arXiv preprint arXiv:2503.17885*, 2025.

APPENDIX

This appendix illustrates how the CWE hierarchical taxonomy enables graduated assessment of LLM security knowledge, demonstrating why our Hierarchical Proximity Score (HPS) metric provides more nuanced evaluation than exact-match accuracy alone.

A. Hierarchical Structure

The CWE hierarchy organizes vulnerabilities across four abstraction levels:

- **Pillar:** Highest-level category (e.g., CWE-664: Improper Control of Resource Lifetime)
- **Class:** Broader weakness category (e.g., CWE-119: Improper Restriction of Operations within Memory Buffer Bounds)
- **Base:** Specific vulnerability type (e.g., CWE-787: Out-of-bounds Write)
- **Variant:** Implementation-specific manifestation (e.g., CWE-120: Buffer Copy without Checking Size of Input)

B. Example: Buffer Overflow Vulnerability

Consider CVE-2022-26490, a buffer overflow vulnerability in the Linux kernel’s NFC driver (Figure 7). The vulnerable code performs an unchecked `memcpy` operation with user-controlled length, while the patched version adds bounds checking.

```
1 // Vulnerable: Direct memcpy with user-controlled length
2 int st21nfca_connectivity_event_received(struct nfc_hci_dev *hdev,
3                                         u8 host, u8 event,
4                                         struct sk_buff *skb) {
5     // Length comes directly from skb->data without validation
6     transaction->params_len = skb->data[ARRAY_SIZE(transaction)];
7     memcpy(transaction->params, &skb->data[ARRAY_SIZE(transaction)],
8           transaction->params_len); // Potential buffer overflow
9 }
10
11 // Patched: Add bounds checking before memcpy
12 int st21nfca_connectivity_event_received(struct nfc_hci_dev *hdev,
13                                         u8 host, u8 event,
14                                         struct sk_buff *skb) {
15     transaction->params_len = skb->data[ARRAY_SIZE(transaction)];
16     if (transaction->params_len > sizeof(transaction->params))
17         return -EINVAL; // Prevent buffer overflow
18     memcpy(transaction->params, &skb->data[ARRAY_SIZE(transaction)],
19           transaction->params_len);
20 }
```

Fig. 7: CVE-2022-26490 (CWE-120): Buffer overflow in NFC driver. The vulnerable code (top) uses user-controlled length without validation; the patch (bottom) adds bounds checking.

An LLM analyzing this vulnerability might predict different hierarchy levels:

- **Exact match** (CWE-120: Buffer Copy without Checking Size) → HPS = 1.0
- **Parent** (CWE-787: Out-of-bounds Write) → HPS = 0.8
- **Grandparent** (CWE-119: Memory Buffer Bounds) → HPS = 0.7
- **Root pillar** (CWE-664: Resource Lifetime) → HPS = 0.4
- **Unrelated** (CWE-284: Access Control) → HPS = 0.0

C. Interpretation

This graduated scoring reveals security reasoning depth: an LLM predicting CWE-119 instead of CWE-120 still demonstrates valuable understanding by recognizing the broader

memory safety issue, even if lacking precision about the specific buffer operation. This approach mirrors how security experts reason—starting with broad categories before drilling down to specifics—enabling more nuanced assessment of LLMs’ security knowledge, from surface-level pattern recognition to fundamental understanding of vulnerability root causes.